

MCTS-MAJ: Memory-Augmented LLM-as-a-Judge with Monte Carlo Tree Search

**Additional Achievement:
Open-Source Research Repository**

Khush Patel

Spring Submission of the thesis in partial fulfillment
of the requirements for the degree of
Bachelor of Science in Computer Science

under the supervision of Professor Bader Rasheed

Innopolis University
2026

Abstract

As an additional achievement for this thesis, the full research artifact has been published as an open-source repository at <https://github.com/khushpatel1102/maj-research>. The repository contains the complete MCTS-MAJ implementation, all benchmark scripts, evaluation results, and documentation required to reproduce the experiments reported in Chapters 3 and 4. This document describes the scope, contents, and public accessibility of the repository, in accordance with the “participate in an open access project” option of the Additional Achievement assignment, as agreed with the supervisor.

1 Project Overview

The repository `maj-research` is a public, open-source research codebase hosting the full implementation of MCTS-MAJ, a memory-augmented LLM-as-a-Judge framework. It was maintained in the open throughout the semester as the research progressed, with a public commit history that reflects the two-stage trajectory described in the thesis: first the MAJ core system, then the MCTS extension with the leakage-free evaluation protocol.

The repository is published under an open-source license and is accessible without authentication, satisfying the “open access project” criterion for the Additional Achievement requirement.

1.1 Repository Link

<https://github.com/khushpatel1102/maj-research>

2 Repository Contents

The repository provides the complete artifact needed to reproduce every result reported in the thesis.

2.1 Source Code

The `src/` directory contains the full Python implementation organized into six modules:

- `models.py`: Pydantic data classes for Policy, Attempt, Issue, Fix, and Semantic nodes, plus an embedding helper.
- `graph_manager.py`: Neo4j connection management, vector index creation, typed CRUD operations, and graph-level retrieval tools.

- `judge.py`: single-pass judge used both statelessly and as the MAJ retrieval-conditioned judge.
- `mcts_judge.py`: MCTS reasoning layer with dynamic subtask generation and four structured-output schemas.
- `mcts_retrieval.py`: MCTS retrieval layer with seven graph-level actions.
- `mcts_pipeline.py`: composition wrappers for the six evaluation modes.

2.2 Benchmark Scripts

Two benchmark runners are provided:

- `benchmark_mcts.py`: runs any of the six evaluation modes on a specified sample size and model. Supports ablations across modes and hyperparameter sweeps.
- `benchmark_leakage_free.py`: implements the leakage-free evaluation protocol with four memory conditions (no memory, self-written, oracle, poisoned at 10 percent, 20 percent, and 50 percent).

2.3 Data and Results

The `data/` directory contains the EvalsBench benchmark CSV files. The `results/` directory contains every per-sample output from every experiment reported in the thesis, one CSV per (mode, condition) pair. This includes the leakage-free stateless, MAJ, MCTS-Judge, and MCTS-Judge + Memory results on 80 test samples, and the poisoned memory runs at each of the three poisoning rates.

2.4 Documentation

The repository root contains a `README.md` describing the setup and invocation of each benchmark, together with several research documents:

- `results/benchmark_summary.md`: summary of all experimental results.
- `update.md`: the Stage 1 progress report covering the seven MAJ experiments.
- `reports/MAJ_Research_Progress_Report.pdf`: formal Stage 1 progress report shared with the supervisor.

2.5 Thesis Chapters

The thesis chapters themselves are also committed to the repository in both LaTeX source and compiled PDF form under the `thesis/` directory, enabling reviewers to match every claim in the written thesis against the exact code that produced it.

3 Reproducibility

Every experiment reported in the thesis can be reproduced from the repository. Specifically:

- The dataset partition is deterministic given a random seed; the default seed is 42 and is fixed in all benchmark scripts.
- All OpenAI model identifiers, MCTS hyperparameters, and retrieval thresholds are set as defaults in the respective modules and explicitly overridable from the command line.
- The Neo4j schema is created automatically on first run through `graph_manager.py`; no manual database setup is required beyond installing Neo4j locally.
- Per-sample CSV outputs allow every reported accuracy number to be re-derived from raw predictions without rerunning the LLM calls.

This end-to-end reproducibility addresses one of the concerns raised in supervisor feedback about research rigor, and it is the primary reason for publishing the repository openly.

4 Public Accessibility and Impact

The repository is hosted on GitHub and visible without authentication. It has been linked from the thesis progress updates sent to the supervisor, making it discoverable by the examining committee and by any researcher who wishes to extend the MCTS-MAJ approach. Commit history is preserved, giving reviewers a full audit trail of how the system evolved over the semester, including the response to supervisor feedback (leakage-free protocol, dynamic subtask generation, oracle and poisoned memory experiments).

By publishing the repository openly, this thesis contributes a reusable research artifact to the community studying LLM-as-a-Judge evaluation, memory-augmented reasoning, and Monte Carlo Tree Search applied to language models.

5 Summary

The open-source repository `khushpatel1102/maj-research` on GitHub is submitted as the Additional Achievement for this thesis. It contains the complete implementation, benchmarks, data, results, documentation, and thesis source files. The repository is public, permissively licensed, and enables end-to-end reproduction of every experiment reported in Chapters 3 and 4. This satisfies the “participate in an open access project” option of the Additional Achievement requirement.